KOMPONENTENBASIERTE ENTWICKLUNG · HTW BERLIN

VueUse

use()

Wiederverwendbare Logik für Vue-Komponenten –
fertig, getestet, typisiert

Hakan Akcan

WiSe 2026/27 · Präsentationsthema 06

Agenda

1. **Das Problem**
   Gleiche Logik in vielen Komponenten
2. **Composables**
   Logik als Funktion – und als Komponente
3. **VueUse im Überblick**
   Was steckt drin, wofür nutzt man es?
4. **Nach VueUse-Vorbild**
   Eigene Composables bauen und testen
5. **Einordnung**
   Vorteile, Nachteile, Faustregel
6. **Live-Demo**
   Vier Beispiele in einer Vue-App

Das Problem: gleiche Logik überall

Kurze Frage: Wer hat schon mal ein Programm geschlossen – und es lief trotzdem im Hintergrund weiter?

Mausposition
Listener an- und abmelden – in jeder Komponente neu.

Dark Mode
Systemeinstellung lesen, Klasse setzen, Auswahl speichern.

LocalStorage
Wert laden, bei Änderung speichern, JSON umwandeln.

Folge: Copy-Paste, vergessene Cleanups (Memory Leaks) und aufgeblähte Komponenten.

Composables: Logik als Funktion

Ein **Composable** ist eine Funktion, die mit der Composition API reaktiven Zustand kapselt.

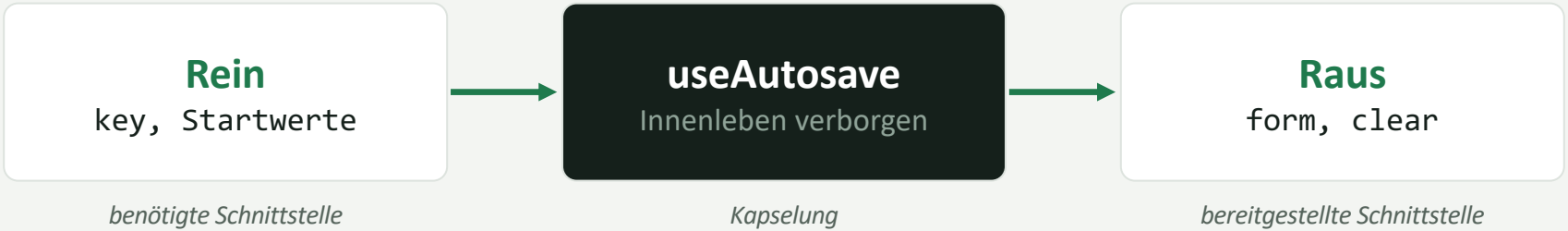
- Name beginnt mit use…
- nutzt ref, computed, Lifecycle
- gibt reaktiven Zustand zurück
- in jeder Komponente nutzbar

Haken: 12 Zeilen – und onUnmounted darf man nie vergessen.

```
// selbst gebaut: useMouse.ts
export function useMouse() {
  const x = ref(0)
  const y = ref(0)
  const update = (e: MouseEvent) => {
    x.value = e.pageX
    y.value = e.pageY
  }
  onMounted(() =>
    window.addEventListener('mousemove', update))
  onUnmounted(() =>
    window.removeEventListener('mousemove', update))
  return { x, y }
}
```

Composable = Komponente ohne UI

Ein Composable ist eigentlich eine Komponente aus Einheit 01 – **nur ohne Oberfläche.**



Ich gebe etwas rein, bekomme etwas raus – wie es drinnen funktioniert, muss die Komponente nicht wissen.

Deshalb kann ich das Innere austauschen, solange rein und raus gleich bleiben.

VueUse: die Werkzeugkiste für Composables

Open-Source-Sammlung fertiger Composables für Vue 3 – gestartet von Anthony Fu (Core-Team von Vue, Vite und Nuxt).

280

Funktionen, davon 221 im Kernpaket

v15

aktuelle Version, Sept. 2026

100 %

in TypeScript typisiert

0

Plugins nötig – nur importieren

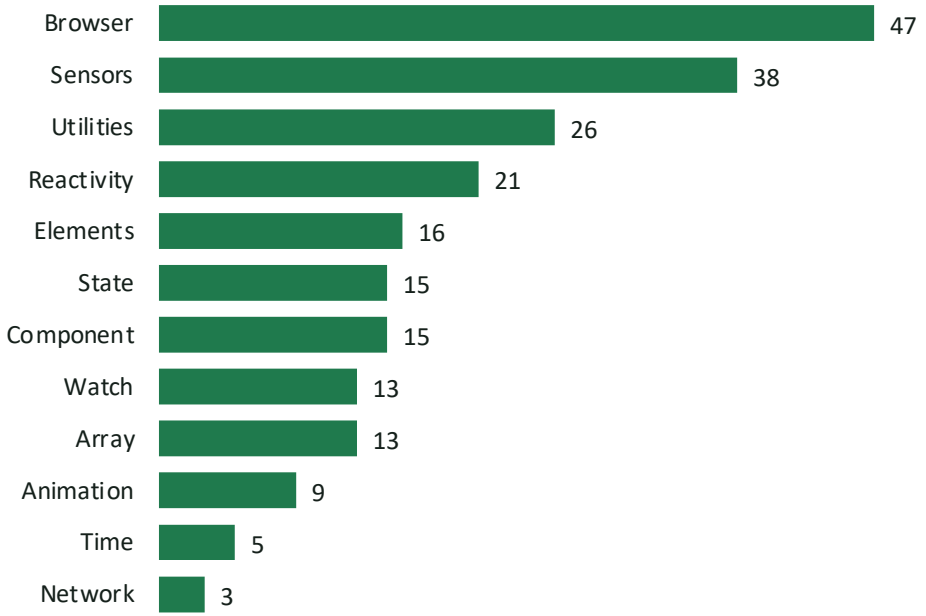
INSTALLIEREN & NUTZEN

```
npm i @vueuse/core
import { useMouse } from '@vueuse/core'
const { x, y } = useMouse()
```

Tree-shakable

Nur importierte Funktionen landen im Bundle.

Wo die 221 Kernfunktionen liegen



Browser + Sensors = 38 %

Der Schwerpunkt: Browser-APIs reaktiv machen.

Browser
`useDark · useClipboard · useTitle`

Sensors
`useMouse · onClickOutside · useOnline`

State
`useLocalStorage · useStorage`

Network
`useFetch · useWebSocket`

Quelle: @vueuse/metadata 15.0.0 (core + shared)

Typische Aufgaben – jeweils eine Zeile

Mausposition verfolgen

```
const { x, y } = useMouse()
```

Dark Mode mit Gedächtnis

```
const isDark = useDark()
```

Wert dauerhaft speichern

```
const name = useLocalStorage('name', '')
```

Popup bei Klick daneben schließen

```
onClickOutside(popup, () => open.value = false)
```

Online-Status anzeigen

```
const online = useOnline()
```

Eigene Composables nach VueUse-Vorbild

1. **Flexible Eingaben**
   MaybeRefOrGetter + toValue(): Wert, Ref oder Getter

2. **Rückgabe als Objekt**
   aus Refs + Funktionen – destrukturierbar und reaktiv

3. **Automatisch aufräumen**
   tryOnScopeDispose statt manuellem onUnmounted

4. **SSR-sicher**
   kein direkter Zugriff auf window beim Import

EIGENES useAutosave (VEREINFACHT)

```
export function useAutosave(key, initial,
  options: { delay?: MaybeRefOrGetter<number> } = {}) {
  // 1) überlebt das Neuladen
  const stored = useLocalStorage(key, initial)
  const form = ref({ ...stored.value })
  // 2) erst speichern, wenn Tipp-Pause
  const save = useDebounceFn(() => {
    stored.value = { ...form.value }
  }, () => toValue(options.delay) ?? 500)
  watch(form, save, { deep: true })
  // 3) Entwurf verwerfen
  const clear = () => {
    form.value = { ...initial }
    stored.value = null
  }
  return { form, clear }
```

Vollständig im Demo-Repo: src/composables/useAutosave.ts

Composables isoliert testen

useAutosave.test.ts · VITEST

```
describe('useAutosave', () => {
  it('speichert erst nach einer Tipp-Pause', async () => {
    vi.useFakeTimers()
    const { form } = useAutosave('key', { text: '' })
    form.value.text = 'Hallo'
    await nextTick()
    vi.advanceTimersByTime(300)
    expect(localStorage.getItem('key')).toBeNull()
    vi.advanceTimersByTime(300)
    await nextTick()
    expect(localStorage.getItem('key')).toContain('Hallo')
  })
  it('stellt den Entwurf nach dem Neuladen wieder her', () => {
    localStorage.setItem('key', '{"text":"Hallo"}')
    const { form } = useAutosave('key', { text: '' })
    expect(form.value.text).toBe('Hallo')
  })
})
```

11 / 11

Tests grün in unter 1 s – ohne Browser, ohne Komponente

Warum so einfach?

Ein Composable ist eine normale Funktion: Eingabe rein, reaktiver Zustand raus. Testbarkeit ist eine Kerneigenschaft einer Komponente (Einheit 01).

Einordnung: wann VueUse wann nicht?

Vorteile

- spart Boilerplate und vergessene Cleanups
- getestet, typisiert, SSR-sicher
- tree-shakable – kleine Bundles
- große Community, Doku mit Live-Beispielen

Nachteile

- zusätzliche Abhängigkeit im Projekt
- versteckt, was im Hintergrund passiert
- Breaking Changes bei Major-Versionen
- viele Funktionen – man muss sie kennen

Faustregel: Für Browser-APIs und Standardaufgaben VueUse nutzen – fachliche Logik als eigenes Composable schreiben.

Live-Demo

1. **Selbst gebaut vs. VueUse**
   `useMouse`
2. **Popup mit Gedächtnis**
   `useLocalStorage` · `onClickOutside`
3. **Browser-APIs reaktiv**
   `useOnline` · `useWindowSize` · `useClipboard`
4. **Formular speichert sich selbst**
   `useAutosave (eigen)` · `useDebounceFn`
5. ✋ **Finale: Steuerung per Handgeste**
   `useUserMedia` · `useRafFn` · `useHandGesture`

Mitmachen
Ruft mir einen Namen für das Popup zu!

Code zum Nachlesen
`github.com/Akcanmuhammedhakan/VueUse`

Fazit

VueUse ist die Standardbibliothek für Composables.



Composables sind Komponenten ohne UI



VueUse liefert 280 fertige davon



Eigene nach dem gleichen Muster bauen

Fragen?

Quellen

vueuse.org/functions
vuejs.org/guide/reusability/composables
npm: @vueuse/core, @vueuse/metadata 15.0.0